

Product Requirements Document: CanonIQ

Product Name

CanonIQ

Full Product Title

CanonIQ: AI-Powered Canonical Mapping Engine

Tagline

Map messy source data into trusted canonical models.

1. Product Summary

CanonIQ is an open-source, local-first, AI-powered canonical schema mapping engine for enterprise data onboarding, validation, transformation, and schema drift detection.

CanonIQ helps teams convert inconsistent source data from customers, vendors, partners, SaaS tools, databases, APIs, files, and internal systems into standardized canonical models. It profiles source datasets, infers field types and patterns, suggests mappings to user-defined canonical schemas, scores mapping confidence, explains mapping decisions, generates validation rules, transforms source data into canonical output, and detects schema drift over time.

CanonIQ is designed to be **domain-agnostic**. It should work for higher education, retail, e-commerce, healthcare, finance, logistics, HR, insurance, marketplaces, B2B SaaS, and AI agent platforms.

The product should be usable as:

1. A **Python SDK**
2. A **CLI tool**
3. A future **server/API mode**
4. A future **review workflow UI**
5. A future **enterprise schema mapping service**

CanonIQ must be professional, extensible, open-source-ready, privacy-conscious, and suitable to present to engineering teams or startups such as Risely.

2. Core Problem

Many companies ingest data from different customers, partners, vendors, or internal systems. These datasets often describe similar entities but use different field names, data types, formats, and conventions.

Example in higher education:

banner_id	-> student_id
student_email	-> email
cumulative_gpa	-> gpa
last_activity_at	-> last_lms_login

Example in retail/e-commerce:

sku_id	-> product_id
item_name	-> product_name
available_qty	-> inventory_quantity
sale_price	-> price

Example in healthcare:

patient_mrn	-> patient_id
dob	-> date_of_birth
diagnosis_code	-> icd_code
visit_time	-> encounter_timestamp

Example in finance:

acct_num	-> account_id
txn_amt	-> transaction_amount
txn_date	-> transaction_date
cust_email	-> customer_email

Example in logistics:

shipment_no	-> shipment_id
tracking_num	-> tracking_number
origin_zip	-> origin_postal_code
delivery_eta	-> estimated_delivery_at

The core problem is:

How do we automatically profile inconsistent source schemas, suggest mappings to canonical models, explain those mappings, validate values, transform source data, and detect schema drift over time?

3. Product Vision

CanonIQ should become a reusable open-source engine for teams that need to onboard messy external or internal data into clean canonical models.

The long-term vision is:

CanonIQ becomes the standard open-source toolkit for canonical schema mapping, customer data onboarding, AI-ready data preparation, and schema drift detection.

It should help companies answer:

1. What fields exist in this customer data?
 2. What do these fields likely mean?
 3. Which canonical fields do they map to?
 4. How confident are we?
 5. Why did the system suggest this mapping?
 6. What validation rules should protect this data?
 7. What changed since the last ingestion?
 8. Is this data ready for downstream analytics, AI agents, ML models, or business workflows?
-

4. Target Users

Primary Users

Data Engineers

Need to profile, validate, transform, and map source data into trusted canonical models.

Analytics Engineers

Need clean, consistent datasets for reporting, dashboards, and semantic layers.

AI Platform Engineers

Need canonical, validated data for AI agents, RAG systems, ML pipelines, and automation workflows.

Customer Implementation Engineers

Need to onboard many customers with different schemas without writing custom code every time.

Startup Engineering Teams

Need lightweight infrastructure for customer data onboarding, schema mapping, and validation.

Secondary Users

1. Solutions architects
 2. Data product managers
 3. Integration engineers
 4. ML engineers
 5. Customer success engineers
 6. Enterprise platform teams
 7. Open-source contributors
-

5. Supported Business Domains

CanonIQ must be designed as a **generic mapping engine**, not an education-only tool.

MVP Domain Examples

The MVP must include examples for:

1. **Higher Education**
2. **Retail / E-commerce**
3. **Healthcare**
4. **Finance**
5. **Logistics**

Future Domain Packs

The architecture should support optional domain packs:

```
canoniq-higher-ed  
canoniq-retail  
canoniq-healthcare  
canoniq-finance  
canoniq-logistics  
canoniq-hr
```

canoniq-insurance
canoniq-marketplace

Each domain pack can include:

1. Canonical schemas
2. Common aliases
3. Validation rules
4. Example datasets
5. Domain-specific matchers
6. Domain-specific documentation

6. Example Domain Use Cases

6.1 Higher Education

Source systems:

SIS
LMS
Canvas
Salesforce
Slate
Degree planning systems
Customer databases

Canonical entities:

Student
Course
Enrollment
DegreePlan
Advisor
AdvisingInteraction
LMSActivity
RiskSignal

AI/business workflows:

Degree planning
Course guidance

Student support
At-risk detection
Advisor outreach
Academic progress analysis

6.2 Retail and E-commerce

Source systems:

Shopify
Amazon Seller Central
ERP
POS
Inventory system
Warehouse management system
Product catalog

Canonical entities:

Product
SKU
Inventory
Order
Customer
Shipment
Promotion
Vendor

Business workflows:

Inventory forecasting
Product catalog normalization
Order analytics
Price optimization
Customer segmentation
Supply chain visibility

6.3 Healthcare

Source systems:

EHR
EMR
Claims systems
Lab systems
Patient intake forms
Scheduling systems

Canonical entities:

Patient
Encounter
Provider
Diagnosis
Procedure
Medication
LabResult
Claim

Business workflows:

Patient record normalization
Care coordination
Claims analysis
Population health analytics
Clinical workflow automation

Important note:

Healthcare examples must use synthetic data only. CanonIQ must not include real patient data.

6.4 Finance

Source systems:

Banking core systems
Transaction feeds
CRM
Payment processors
Ledger systems
Risk systems

Canonical entities:

Customer
Account
Transaction
Payment
Merchant
Loan
RiskProfile
LedgerEntry

Business workflows:

Transaction normalization
Fraud analytics
Customer 360
Risk scoring
Financial reporting

6.5 Logistics and Supply Chain

Source systems:

TMS
WMS
Carrier APIs
ERP
Tracking systems
Supplier portals

Canonical entities:

Shipment
Carrier
Package
Warehouse
Route
DeliveryEvent
InventoryMovement
Supplier

Business workflows:

Shipment tracking
ETA prediction
Inventory movement analysis
Carrier performance
Supply chain visibility

6.6 HR and Workforce

Source systems:

HRIS
Payroll
Applicant tracking systems
Learning management systems
Performance systems

Canonical entities:

Employee
Candidate
Role
Department
PayrollRecord
PerformanceReview
TrainingRecord

Business workflows:

Employee data normalization
Workforce analytics
Talent pipeline reporting
Compliance reporting

7. Product Goals

CanonIQ should:

1. Profile source datasets.
2. Load user-defined canonical schemas.
3. Suggest source-to-canonical field mappings.

4. Score mapping confidence.
 5. Explain why each mapping was suggested.
 6. Generate validation rules.
 7. Transform source data into canonical output.
 8. Detect schema drift over time.
 9. Support multiple business domains.
 10. Support both CLI and Python SDK usage.
 11. Be local-first by default.
 12. Be open-source ready with documentation, tests, examples, and license.
 13. Be extensible through plugins/domain packs.
 14. Support optional AI-assisted mapping without requiring external AI by default.
-

8. Non-Goals for MVP

The MVP should not build:

1. A full production web UI.
 2. A hosted SaaS product.
 3. Complex authentication or RBAC.
 4. Mandatory external LLM calls.
 5. Real-time streaming ingestion.
 6. Full Spark support.
 7. Full enterprise data catalog integration.
 8. Full database connector framework.
 9. Complex derived field transformations.
 10. Multi-user approval workflows.
-

9. Product Principles

9.1 Domain-Agnostic Core

CanonIQ core should not be hardcoded for education, retail, healthcare, or any specific industry.

The core engine should work with generic concepts:

```
SourceDataset
SourceField
CanonicalSchema
CanonicalEntity
CanonicalField
MappingSuggestion
```

```
ValidationRule
DriftReport
```

Domain-specific logic should be provided through configuration or optional domain packs.

9.2 Local-First by Default

CanonIQ must not send source data, schemas, sample values, or mappings to external APIs by default.

9.3 Optional AI

AI-assisted mapping should be optional and pluggable.

MVP should implement rule-based matching and provide an optional AI adapter interface.

9.4 Explainability

Every mapping suggestion must include reasons.

9.5 Safe Automation

High-confidence mappings can be auto-approved. Medium-confidence mappings require review. Low-confidence mappings should remain unmapped.

Suggested thresholds:

```
0.90 - 1.00: auto_approved
0.70 - 0.89: requires_review
below 0.70: low_confidence
```

9.6 Versioned Canonical Models

Canonical schemas should include domain, entity, and version.

Example:

```
domain: retail
entity: product
version: 1
```

9.7 Open-Source Professionalism

The repository must include:

```
README.md
LICENSE
CONTRIBUTING.md
CODE_OF_CONDUCT.md
SECURITY.md
CHANGELOG.md
pyproject.toml
tests
examples
docs
GitHub Actions CI
Dockerfile
```

10. MVP Feature Scope

MVP Version

v0.1.0

Required MVP Features

1. CSV profiling
2. JSON profiling
3. Canonical schema YAML loader
4. Field type inference
5. Alias-based matching
6. Name similarity matching
7. Type matching
8. Pattern matching
9. Confidence scoring
10. Mapping explanations
11. Validation rule generation
12. Source-to-canonical transformation
13. Schema drift detection
14. CLI commands
15. Python SDK interface
16. Higher education example
17. Retail/e-commerce example
18. Healthcare example

19. Finance example
 20. Logistics example
 21. Unit tests
 22. README and docs
 23. Security policy
 24. Contribution guide
 25. GitHub Actions CI
 26. Dockerfile
-

11. Supported Input Types for MVP

Required

1. CSV
2. JSON records

Optional

1. JSON Lines

Future

1. SQL tables
 2. Parquet
 3. API responses
 4. Spark DataFrames
 5. Polars DataFrames
 6. Pandas DataFrames directly
-

12. Supported Output Types for MVP

1. Profile JSON
 2. Mapping suggestions JSON
 3. Validation rules YAML
 4. Canonical CSV output
 5. Drift report JSON
-

13. Repository Structure

Create this repository structure:

```
canoniq/
├── canoniq/
│   ├── __init__.py
│   ├── engine.py
│   ├── config.py
│   ├── core/
│   │   ├── __init__.py
│   │   ├── models.py
│   │   └── constants.py
│   ├── profiler/
│   │   ├── __init__.py
│   │   ├── csv_profiler.py
│   │   ├── json_profiler.py
│   │   └── type_inference.py
│   ├── registry/
│   │   ├── __init__.py
│   │   ├── canonical_schema.py
│   │   └── mapping_registry.py
│   ├── matcher/
│   │   ├── __init__.py
│   │   ├── name_matcher.py
│   │   ├── alias_matcher.py
│   │   ├── type_matcher.py
│   │   ├── pattern_matcher.py
│   │   └── mapping_engine.py
│   ├── scoring/
│   │   ├── __init__.py
│   │   └── confidence.py
│   ├── validation/
│   │   ├── __init__.py
│   │   ├── rule_generator.py
│   │   └── validator.py
│   ├── transform/
│   │   ├── __init__.py
│   │   └── transformer.py
│   └── drift/
```

```

├── __init__.py
├── drift_detector.py
├── ai/
│   ├── __init__.py
│   ├── base.py
│   └── optional_ai_matcher.py
├── domains/
│   ├── __init__.py
│   ├── higher_ed.py
│   ├── retail.py
│   ├── healthcare.py
│   ├── finance.py
│   └── logistics.py
├── cli.py
├── examples/
│   ├── higher_ed/
│   │   ├── canonical_student.yml
│   │   ├── source_students.csv
│   │   ├── new_source_students.csv
│   │   └── demo.py
│   ├── retail/
│   │   ├── canonical_product.yml
│   │   ├── source_products.csv
│   │   ├── new_source_products.csv
│   │   └── demo.py
│   ├── healthcare/
│   │   ├── canonical_patient.yml
│   │   ├── source_patients.csv
│   │   ├── new_source_patients.csv
│   │   └── demo.py
│   ├── finance/
│   │   ├── canonical_transaction.yml
│   │   ├── source_transactions.csv
│   │   ├── new_source_transactions.csv
│   │   └── demo.py
│   └── logistics/
│       ├── canonical_shipment.yml
│       └── source_shipments.csv

```

```
|
|   |   | new_source_shipments.csv
|   |   | demo.py
|
| tests/
|   | test_csv_profiler.py
|   | test_json_profiler.py
|   | test_type_inference.py
|   | test_canonical_schema.py
|   | test_mapping_engine.py
|   | test_transformer.py
|   | test_validation_rules.py
|   | test_drift_detector.py
|   | test_domain_examples.py
|
| docs/
|   | architecture.md
|   | quickstart.md
|   | concepts.md
|   | domain_packs.md
|   | roadmap.md
|   | higher_ed_use_case.md
|   | retail_use_case.md
|   | healthcare_use_case.md
|   | finance_use_case.md
|   | logistics_use_case.md
|
| .github/
|   | workflows/
|   |   | ci.yml
|
| README.md
| LICENSE
| CONTRIBUTING.md
| CODE_OF_CONDUCT.md
| SECURITY.md
| CHANGELOG.md
| pyproject.toml
| .gitignore
| Dockerfile
```


14. Technology Stack

Language

Python 3.10+

Required Dependencies

```
pandas  
pyyaml  
pydantic  
typer  
rich  
rapidfuzz  
pytest
```

Optional Future Dependencies

```
polars  
sentence-transformers  
openai  
anthropic  
duckdb  
pyarrow
```

Do not require optional AI dependencies for core installation.

15. Core Data Models

The core code should use generic data models.

SourceFieldProfile

```
class SourceFieldProfile:  
    name: str  
    inferred_type: str  
    null_rate: float  
    unique_rate: float  
    sample_values: list  
    patterns: list[str]
```

```
min: float | None
max: float | None
```

CanonicalField

```
class CanonicalField:
    name: str
    type: str
    required: bool
    description: str | None
    aliases: list[str]
    min: float | None
    max: float | None
```

MappingSuggestion

```
class MappingSuggestion:
    source_field: str
    canonical_field: str
    confidence: float
    status: str
    reasons: list[str]
    signals: dict
```

ValidationRule

```
class ValidationRule:
    field: str
    rule: str
    severity: str
    params: dict
```

DriftReport

```
class DriftReport:
    status: str
    missing_fields: list[str]
    new_fields: list[str]
    type_changes: list[dict]
    suggested_remappings: list[dict]
```

16. Canonical Schema YAML Format

Canonical schemas must be domain-flexible.

Example general format:

```
domain: retail
entity: product
version: 1

fields:
  product_id:
    type: string
    required: true
    description: Unique product identifier
    aliases:
      - sku
      - sku_id
      - item_id
      - product_sku

  product_name:
    type: string
    required: true
    description: Product display name
    aliases:
      - item_name
      - title
      - product_title

  price:
    type: decimal
    required: false
    min: 0.0
    aliases:
      - sale_price
      - unit_price
      - list_price

  inventory_quantity:
    type: integer
    required: false
    min: 0
    aliases:
```

- available_qty
- quantity_available
- stock_on_hand

17. Required Example Canonical Schemas

17.1 Higher Education: Student

```
domain: higher_ed
entity: student
version: 1

fields:
  student_id:
    type: string
    required: true
    aliases:
      - banner_id
      - emplid
      - person_uid
      - student_number

  email:
    type: email
    required: true
    aliases:
      - student_email
      - email_address
      - primary_email

  gpa:
    type: decimal
    required: false
    min: 0.0
    max: 4.0
    aliases:
      - cumulative_gpa
      - overall_gpa
      - term_gpa

  last_lms_login:
    type: timestamp
    required: false
    aliases:
```

- last_activity_at
- canvas_last_seen
- last_login_date

17.2 Retail: Product

```
domain: retail
entity: product
version: 1

fields:
  product_id:
    type: string
    required: true
    aliases:
      - sku
      - sku_id
      - item_id
      - product_sku

  product_name:
    type: string
    required: true
    aliases:
      - item_name
      - product_title
      - title

  price:
    type: decimal
    required: false
    min: 0.0
    aliases:
      - sale_price
      - unit_price
      - list_price

  inventory_quantity:
    type: integer
    required: false
    min: 0
    aliases:
      - available_qty
      - stock_qty
      - stock_on_hand
```

17.3 Healthcare: Patient

Use synthetic fields only.

```
domain: healthcare
entity: patient
version: 1

fields:
  patient_id:
    type: string
    required: true
    aliases:
      - patient_mrn
      - mrn
      - patient_number

  date_of_birth:
    type: date
    required: false
    aliases:
      - dob
      - birth_date

  email:
    type: email
    required: false
    aliases:
      - patient_email
      - contact_email

  diagnosis_code:
    type: string
    required: false
    aliases:
      - icd_code
      - dx_code
      - diagnosis
```

17.4 Finance: Transaction

```
domain: finance
entity: transaction
version: 1
```

```
fields:
  transaction_id:
    type: string
    required: true
    aliases:
      - txn_id
      - transaction_ref
      - payment_id

  account_id:
    type: string
    required: true
    aliases:
      - acct_num
      - account_number
      - acct_id

  transaction_amount:
    type: decimal
    required: true
    aliases:
      - txn_amt
      - amount
      - payment_amount

  transaction_date:
    type: timestamp
    required: true
    aliases:
      - txn_date
      - payment_date
      - posted_at
```

17.5 Logistics: Shipment

```
domain: logistics
entity: shipment
version: 1

fields:
  shipment_id:
    type: string
    required: true
    aliases:
      - shipment_no
      - shipment_number
```

```
- load_id

tracking_number:
  type: string
  required: false
  aliases:
    - tracking_num
    - carrier_tracking_id

carrier:
  type: string
  required: false
  aliases:
    - carrier_name
    - shipping_provider

estimated_delivery_at:
  type: timestamp
  required: false
  aliases:
    - delivery_eta
    - eta
    - expected_delivery_time
```

18. CLI Requirements

The CLI must support generic domain examples.

Profile

```
canoniq profile --source examples/retail/source_products.csv --out profile.json
```

Suggest

```
canoniq suggest
--profile profile.json
--canonical examples/retail/canonical_product.yml
--out suggestions.json
```


Rules

```
canoniq rules
  --suggestions suggestions.json
  --canonical examples/retail/canonical_product.yml
  --out validation_rules.yml
```

Apply

```
canoniq apply
  --source examples/retail/source_products.csv
  --mapping suggestions.json
  --out canonical_products.csv
```

Drift Check

```
canoniq drift-check
  --source examples/retail/new_source_products.csv
  --mapping suggestions.json
  --canonical examples/retail/canonical_product.yml
  --out drift_report.json
```

Demo

```
canoniq demo higher-ed
canoniq demo retail
canoniq demo healthcare
canoniq demo finance
canoniq demo logistics
```

The demo command must support all provided domains.

19. Python SDK Requirements

Implement a domain-agnostic SDK.

```
from canoniq import CanonIQ
```

```
engine = CanonIQ()

profile = engine.profile_csv("examples/retail/source_products.csv")

suggestions = engine.suggest_mappings(
    profile=profile,
    canonical_schema_path="examples/retail/canonical_product.yml"
)

rules = engine.generate_validation_rules(
    suggestions=suggestions,
    canonical_schema_path="examples/retail/canonical_product.yml"
)

canonical_df = engine.apply_mapping(
    source_path="examples/retail/source_products.csv",
    mapping=suggestions
)

drift_report = engine.detect_drift(
    source_path="examples/retail/new_source_products.csv",
    previous_mapping=suggestions,
    canonical_schema_path="examples/retail/canonical_product.yml"
)
```

The SDK should work the same way for any domain as long as a canonical schema is provided.

20. Matching Engine Requirements

The mapping engine should combine these signals:

1. Alias match
2. Name similarity
3. Type compatibility
4. Pattern match
5. Range compatibility
6. Domain context, if provided
7. Optional semantic score, future

Confidence Formula

Use:

```
confidence =  
  alias_score * 0.40 +  
  name_score * 0.20 +  
  type_score * 0.15 +  
  pattern_score * 0.15 +  
  range_score * 0.10
```

Clamp score between 0.0 and 1.0.

Status Rules

```
confidence >= 0.90 -> auto_approved  
0.70 <= confidence < 0.90 -> requires_review  
confidence < 0.70 -> low_confidence
```

21. Type Inference Requirements

Implement type inference for:

```
string  
integer  
decimal  
email  
timestamp  
date  
boolean  
text  
unknown
```

Rules:

1. Email regex -> email
 2. Integer parse -> integer
 3. Float parse -> decimal
 4. Common date formats -> date
 5. Common datetime formats -> timestamp
 6. true/false/yes/no/0/1 -> boolean
 7. Long average text length -> text
 8. Else string
-

22. Pattern Detection Requirements

Detect:

```
email
identifier_like
uuid_like
decimal
integer
range_0_to_4
positive_number
timestamp_iso
date
mostly_unique
enum_like
high_null_rate
currency_like
postal_code_like
phone_like
```

Patterns should be useful across many domains.

23. Validation Rule Generation

Generate validation rules from canonical schema and profile.

Examples:

1. Required field -> not_null
 2. Email field -> valid_email
 3. Decimal with min/max -> range
 4. Integer with min/max -> range
 5. Timestamp field -> valid_timestamp
 6. Date field -> valid_date
 7. Mostly unique ID field -> unique
 8. Enum-like values -> allowed_values, optional
-

24. Transformation Requirements

The apply command should:

1. Load source data.
 2. Load mapping suggestions.
 3. Use mappings with status:
 4. auto_approved
 5. approved
 6. Rename mapped source columns to canonical fields.
 7. Drop unmapped fields by default.
 8. Write canonical CSV.
 9. Preserve data types when possible.
 10. Add optional metadata columns in future.
-

25. Drift Detection Requirements

Drift detection should be generic across domains.

Detect:

1. Previously mapped source field missing
2. New source field added
3. Type changed
4. Required canonical field no longer mapped
5. Suggested remapping available

Example retail drift:

Old:

```
sku_id -> product_id
available_qty -> inventory_quantity
```

New:

```
product_sku
stock_on_hand
```

Suggested remap:

```
product_sku -> product_id  
stock_on_hand -> inventory_quantity
```

26. Documentation Requirements

README.md

Must position CanonIQ as domain-agnostic.

Include use cases for:

1. Higher education
2. Retail/e-commerce
3. Healthcare
4. Finance
5. Logistics
6. SaaS customer onboarding
7. AI agent platforms

docs/domain_packs.md

Explain how to add a new domain.

Required steps:

1. Create canonical schema YAML.
2. Add aliases.
3. Add synthetic example source data.
4. Add tests.
5. Add demo command support.

docs/risely_use_case.md or docs/higher_ed_use_case.md

Keep Risely/higher-ed as one example only, not the whole product.

27. Security Requirements

Create `SECURITY.md`.

Include:

CanonIQ is local-first by default. CanonIQ does not send source data, schemas, sample values, or mappings to external AI providers unless the user explicitly configures an external AI adapter.

Additional requirements:

1. No telemetry in MVP.
2. No external API calls in core package.
3. No real healthcare, financial, education, or personal data in examples.
4. All examples must be synthetic.
5. No API keys in repo.

28. Testing Requirements

Tests must cover at least:

1. Higher education mapping
2. Retail mapping
3. Healthcare mapping
4. Finance mapping
5. Logistics mapping
6. CSV profiling
7. JSON profiling
8. Type inference
9. Mapping confidence
10. Validation rule generation
11. Transformation
12. Drift detection

29. Acceptance Criteria

The project is complete when these commands work:

```
pip install -e .
pytest
canoniq demo higher-ed
canoniq demo retail
canoniq demo healthcare
```

```
canoniq demo finance
canoniq demo logistics
```

Each demo must:

1. Profile source data.
2. Suggest mappings.
3. Generate validation rules.
4. Apply mappings.
5. Detect drift.
6. Write outputs to an output folder.
7. Print a clean success summary.

30. Build Instructions for Claude Code

Build CanonIQ as a production-quality open-source Python project.

Important: Do not make CanonIQ education-specific. Higher education should be one domain example only.

Prioritize the following:

1. Create project structure.
2. Create pyproject.toml.
3. Implement generic core models.
4. Implement canonical schema loader.
5. Implement CSV profiler.
6. Implement JSON profiler.
7. Implement type inference.
8. Implement alias/name/type/pattern/range matchers.
9. Implement mapping engine and confidence scoring.
10. Implement validation rule generator.
11. Implement transformer.
12. Implement drift detector.
13. Implement CanonIQ SDK class.
14. Implement Typer CLI.
15. Add higher-ed, retail, healthcare, finance, and logistics examples.
16. Add unit tests for all domains.
17. Add README and docs.
18. Add SECURITY.md, CONTRIBUTING.md, CODE_OF_CONDUCT.md, CHANGELOG.md.
19. Add GitHub Actions CI.
20. Add Dockerfile.

The MVP should be local-first, privacy-conscious, typed, tested, modular, and ready to push to GitHub.

31. Final Deliverable

The final deliverable should be a working open-source repository where this works:

```
pip install -e .  
pytest  
canoniq demo higher-ed  
canoniq demo retail  
canoniq demo healthcare  
canoniq demo finance  
canoniq demo logistics
```

The final project should be professional enough to present to startups, engineering teams, open-source contributors, and companies that need data onboarding and canonical schema mapping across multiple business domains.